

Programa AI-First · Fase 1

Jikkosoft — Preparación y Reto

Preparado por Diego Trujillo · diego.trujillo@jikkosoft.com

Aspectos importantes

Por qué Hermes, Jikkosoft pasa de escribir código a orquestar contexto. La investigación interna (hermes-exploratory, 2026) lo confirma con datos: mejorar la spec es 3× más efectivo que actualizar el modelo — pasar de Spec A (~150 palabras) a Spec B (~480 palabras) produce +26 puntos promedio en calidad de output; subir de Haiku a Opus en la misma spec produce solo +12 a +24. Una spec mal escrita hace que el modelo más costoso alucine convenciones e ignore reglas de integridad. Una spec bien escrita hace que el modelo más barato produzca output de producción. Hermes es el canal obligatorio que hace este proceso repetible, trazable y transferible.

Requisitos mínimos

Herramienta	Instalación / Acceso
Codex CLI	Instalar desde github.com/openai/codex
Hermes agent (NousResearch)	github.com/NousResearch/hermes-agent — install.sh bash · verificar: hermes --help && hermes doctor
Git	git-scm.com · macOS/Linux: preinstalado · Windows: descargar instalador
GPG key	<code>gpg --full-generate-key</code> (RSA 4096, sin expiración, email de trabajo) — requerido para acceso git-crypt a archivos internos del repo
API key LLM	Contactar a Juan David para obtener acceso a una API
Cuenta GitHub o GitLab	github.com o gitlab.com — repo público donde alojar la entrega del reto

Principio fundacional

Todo proyecto AI empieza con /init. Antes de escribir una sola spec, antes de tocar código, antes de todo: ejecuta `claude /init` en el repositorio. Esto genera el `CLAUDE.md` base y establece el contrato AI del proyecto. Sin este paso, la IA trabaja sin contexto y los resultados son impredecibles.

Estructura general

Etapa	Descripción	Fechas
1 — Capacitación	Codex CLI · Spec Engineering · CLAUDE.md · Plan & Loop · MCP Servers · Subagentes · Git	24-25 jun
2 — Adaptabilidad	Construcción y entrega del proyecto (Reto Fase 1: DEV + QA en paralelo)	26 jun – 6 jul
3 — Adaptabilidad con lo actual	Integración del workflow AI-first con proyectos y contexto existente	hasta 7 jul

Evaluación: viernes 7 de julio · demo de 5-7 min por track.

Semana 1 — Codex CLI

Workshop 1 — Apertura + Codex CLI

Lunes

- Apertura del programa: contexto del “momento cero” y el nuevo modelo operativo AI-first.
- Instalación y configuración: Claude Code + modelo conectado a Hermes.
- El ritual /init en vivo sobre un repo vacío: qué genera, por qué importa.
- Navegación básica: slash commands, permisos, modo interactivo vs. autónomo.
- Hermes como intermediario obligatorio: toda interacción con modelos pasa por aquí. Libertad de elegir uno o varios LLMs para codificar o implementar (ej: Claude Sonnet, Haiku, Opus, Codex, DeepSeek, Kimi K2, etc.).

Referencia de comandos Hermes:

Comando	Uso
hermes setup	Configuración inicial — conecta proveedores (Anthropic, DeepSeek, Moonshot)
hermes doctor	Diagnóstico de instalación y conectividad
hermes model	Selector interactivo de modelo (muestra todos los disponibles)
/model anthropic:claude-sonnet-4-6	Cambia el modelo activo dentro del TUI
/new	Abre conversación fresca — evita memory bleed entre tareas
hermes config list	Lista las claves de config aceptadas por la versión instalada

Workshop 2 — Spec Engineering

Jueves o viernes

- Por qué la calidad de la spec supera la elección del modelo.
- La matriz real: 3 niveles de spec × 7 modelos — caso de estudio extraído del experimento interno de Diego (hermes-exploratory).
- Hallazgo clave: Haiku + Spec C (88 pts) > Opus + Spec A (72 pts). Spec B + Sonnet es el punto de equilibrio costo/calidad.
- Ejercicio práctico: cada participante escribe 3 versiones de spec para algo de su contexto y compara outputs.

Checklist de una spec efectiva (nivel Spec B — punto de equilibrio):

- Dominio — qué representa la spec, en 2-3 oraciones.
- Scope — lista explícita de entidades/tablas con columnas clave.
- Tech stack — versión de DB, estrategia de IDs, formato de timestamps.
- Convenciones — naming, columnas de estado (CHECK vs ENUM), tipo de moneda.
- Reglas de integridad — soft delete, ON DELETE, UNIQUE, CHECK constraints.
- Reglas de safe-change — columnas nullable, sin renombrados, índices en FKs.
- Fuera de scope — qué NO generar (evita tablas alucinadas).
- Entregable esperado — formato exacto de output (solo SQL, sin prosa, sin fences).

Rubric de evaluación de output (0-100):

Categoría	Máx	Qué evalúa
Structure	30	Tablas requeridas, FKs, sin tablas inventadas
Naming	15	snake_case, id/{table}_id, created_at/updated_at
Integrity	20	PK/FK, NOT NULL, UNIQUE, soft delete, CASCADE/RESTRICT
Comments	15	Tablas y decisiones no obvias documentadas
Query feasibility	10	Queries clave soportados, índices en FKs y hot paths
Spec adherence	10	Siguió la spec, sin features inventadas

Regla práctica: score < 80 → la spec necesita más detalle, no un modelo más caro. Referencia completa: github.com/diegotrujillo-jikko/hermes-exploratory

Días sin workshop: adaptación libre + punto de control (3 líneas de avance + bloqueos vía Hermes).

Semana 2 — CLAUDE.md

Workshop 3 — Anatomía del CLAUDE.md

Lunes

- El /init genera el esqueleto; tú lo conviertes en un contrato AI preciso.
- Componentes esenciales: authority hierarchy, safe/unsafe zones, naming conventions, model selection table.
- Revisión en vivo de un CLAUDE.md real de producción (21 KB, API de SILIN).
- Diferencia entre un README y un AI operating contract.

Workshop 4 — CLAUDE.md hands-on

Jueves o viernes

- Cada participante toma su repo actual (SILIN / integraciones) y escribe su propio CLAUDE.md desde cero.
- Track DEV: orientado a construcción (safe zones para schema, API, FE).
- Track QA: orientado a pruebas (qué no modificar del SUT, cómo describir el alcance de cobertura).
- Revisión cruzada en pares.

Semana 3 — Plan & Loop · MCP Servers · Subagentes · Git

Workshop 5 — Plan Mode + Loop + Skills

Lunes

- /plan antes de cualquier tarea compleja: cómo estructurar el trabajo antes de generar código.
- Loop modes para tareas iterativas.
- Sistema everything-claude-code: el skill correcto para cada tipo de tarea.

Skill	Uso
:tdd-workflow	Desarrollo guiado por pruebas
:backend-patterns	APIs y servicios
:security-review	Revisión de seguridad
:database-reviewer	Schema y queries

- Demostración completa: /init → /plan → build con skills apropiados.

Workshop 6 — MCP Servers + Subagentes + Git + Cierre

Jueves o viernes

- MCP Servers: Figma como fuente de diseño leíble por IA · Azure DevOps WI como PRD · claude-mem para memoria persistente entre sesiones.
- Subagentes paralelos: el patrón LLM-as-judge — un modelo genera el output, otro lo evalúa contra un rubric. Caso real implementado en hermes-exploratory Phase 2: spec → Hermes (generator) → SQL → Hermes (judge) → score 0-100 → comentario en PR → gate (precision ≥ 0.85 = merge ✓, si no → itera la spec). El gate bloquea el merge hasta que el output pase el umbral, sin intervención manual.
- Git con AI: commits semánticos, PRs, ai-dev-log para trazabilidad de sesiones.
- Cierre del curso y lanzamiento oficial de los Retos Fase 1.

Semana 4 — Reto Fase 1

Dos tracks en paralelo

Track DEV — Implementa una app similar a mini-tienda-base

Misión: En 4 días, implementar una app con la misma estructura que mini-tienda-base — frontend, backend, base de datos e integración — sobre un caso de uso comercial distinto a SILIN, sin escribir código a mano. Todo se genera y se itera a través de Hermes como agente principal y uno o varios LLMs de su preferencia para codificar o implementar (ej: Claude Sonnet, Haiku, Opus, Codex, DeepSeek, Kimi K2, etc.).

Componentes obligatorios (misma estructura que mini-tienda-base):

Componente	Requisito
Backend	Servicio con lógica de negocio y API REST
Frontend	Interfaz web funcional (equivalente a static/index.html)
Base de datos	PostgreSQL, SQLite, MongoDB o la de tu preferencia
Integración	Al menos una integración externa o entre módulos propios

Reglas:

1. Cero código manual — solo especificas, diriges, revisas e iteras.
2. Hermes como agente principal y uno o varios LLMs de su preferencia para codificar o implementar (ej: Claude Sonnet, Haiku, Opus, Codex, DeepSeek, Kimi K2, etc.) — obligatorio e innegociable.
3. Caso de uso libre — distinto a SILIN y distinto a mini-tienda.
4. Repositorio público (GitHub o GitLab).

Track QA — Diseña y ejecuta una estrategia de pruebas

Misión: En 4 días, diseñar, automatizar y ejecutar una estrategia de pruebas E2E sobre la aplicación base provista (mini-tienda-base), sin escribir scripts a mano. Se usa Hermes como agente principal y uno o varios LLMs de su preferencia para codificar o implementar (ej: Claude Sonnet, Haiku, Opus, Codex, DeepSeek, Kimi K2, etc.).

Requisitos mínimos de entrega:

Componente	Requisito
Plan de pruebas	Estrategia, alcance, riesgos y criterios de aceptación

Pruebas funcionales	Casos: happy path, borde y negativos
Automatización UI / E2E	Suite automatizada sobre el frontend
Pruebas de API	Contratos, estados HTTP, errores
Pruebas de datos e integración	Validación de BD y de la integración de precios
Reporte de defectos	Hallazgos con severidad, pasos de reproducción y evidencia

Reglas:

1. Cero scripts manuales — la IA los genera, tú especificas y revisas.
2. Hermes como agente principal y uno o varios LLMs de su preferencia para codificar o implementar (ej: Claude Sonnet, Haiku, Opus, Codex, DeepSeek, Kimi K2, etc.) — obligatorio como agente principal.
3. No modificar el código de producción del SUT.
4. Repositorio público (GitHub o GitLab).

Cronograma — Semana 4

Día	DEV	QA
Lunes	/init → CLAUDE.md → primera spec → inicio de construcción	/init → plan de pruebas → setup Playwright / pytest
Martes-Jueves	Backend + Frontend + DB + Integración · punto de control diario (3 líneas vía Hermes)	E2E + API + datos + escenario PRICING_FAIL=1 · punto de control diario
Viernes AM	Repo final + HERMES_CONTEXT.md	Repo final + HERMES_CONTEXT.md
Viernes PM	Demo 5-7 min (viernes de la semana del reto) + cata por grupos	Demo 5-7 min (viernes de la semana del reto) + cata por grupos

Entregables de ambos tracks

- HERMES_CONTEXT.md (obligatorio, tan importante como el código)

Track DEV:

- Qué construiste y qué problema resuelve
- Stack y arquitectura: componentes y cómo se conectan
- Cómo usaste Hermes y los LLMs: skills, specs y prompts que mejor funcionaron
- Decisiones y trade-offs
- Bloqueos y cómo los resolviste
- Qué mejorarías o pedirías
- Enlace al repositorio

Track QA:

- Alcance: qué probaste y bajo qué estrategia
- Enfoque y herramientas: tipos de prueba y stack usado
- Cómo usaste Hermes y los LLMs: skills, specs e iteraciones
- Decisiones y trade-offs: qué priorizaste y por qué
- Bloqueos y cómo los resolviste
- Hallazgos principales / riesgos detectados
- Enlace al repositorio

Criterios de evaluación

Dimensión	Peso
Calidad del HERMES_CONTEXT.md y trazabilidad del proceso	25%
Autonomía: investigó y desbloqueó por cuenta propia	25%
Orquestación de IA: claridad de specs, iteración, manejo de contexto	18%
Funcionalidad E2E (DEV) / Cobertura y profundidad (QA)	14%
Previsión y comunicación a tiempo	10%
Criterio técnico (DEV) / Criterio de QA (QA)	8%

Se busca criterio y adaptación, no el proyecto más grande ni la suite más extensa.

Sobre créditos y costos

Los modelos tienen costo y las capas gratuitas se agotan. Prever esto es parte del reto.

Si necesitas acceso a un modelo corporativo o te vas a quedar corto de créditos, levanta la mano a tiempo — no el último día. Anticiparte y comunicar a tiempo es una de las capacidades que se están evaluando.

Dudas durante el programa: canalízalas a Diego Trujillo — diego.trujillo@jikkosoft.com o Google Chat.